CYNBIT AUTOBLUEPRINT: AI Patient Symptom Triage & Clinical Intake Platform

Client Organization: MedAxis Multi-Specialty Clinics | **Contact:** Dr. Aravind Mehta

Industry: Healthcare & Clinical AI | **Governance Status:** `PENDING_APPROVAL`

Critic Quality Score: `99/100` | **Requirement Coverage:** `100.0%`

1. Project Overview

This Technical Blueprint defines the proposed architecture, technology stack, data model, and phased implementation plan for 'AI Patient Symptom Triage & Clinical Intake Platform' requested by MedAxis Multi-Specialty Clinics (Dr. Aravind Mehta). Classified under 'Healthcare & Clinical AI Systems', the solution aligns with Cynbit Technologies' 6-stage architectural lifecycle and Human-in-the-Loop (HITL) engineering governance.

2. Customer Business Problem

MedAxis Multi-Specialty Clinics (Healthcare & Clinical AI) seeks to solve: We operate 14 outpatient clinics and experience severe front-desk triage bottlenecks. We need an AI diagnostic triage assistant that captures structured patient symptoms before arrival, classifies urgency tiers, integrates with our PostgreSQL EHR records, and routes high-risk cas...

3. Project Objectives

- Reduce patient intake wait times by 50%
- ensure 100% physician oversight on high-risk triage classifications
- maintain 99
- 9% clinical system availability

4. Extracted Functional Requirements

- **FR-01**: Implement capability: We operate 14 outpatient clinics and experience severe front-desk triage bottlenecks *(Mapping: Mapped to Core Application Service & Workflow Module #1)*
- **FR-02**: Implement capability: We need an AI diagnostic triage assistant that captures structured patient symptoms before arrival, classifies urgency tiers, integrates with our PostgreSQL EHR records, and routes high-risk cases to a physician Human-in-the-Loop dashboard for immediate sign-off *(Mapping: Mapped to Core Application Service & Workflow Module #2)*
- **FR-03**: Implement capability: Strict PHI data encryption, role-based access control for doctors vs *(Mapping: Mapped to Core Application Service & Workflow Module #3)*
- **FR-04**: Implement capability: receptionists, and full audit trails *(Mapping: Mapped to Core Application Service & Workflow Module #4)*
- **FR-05**: Structured AI symptom/request intake and priority triage routing engine with human clinician/operator review. *(Mapping: Mapped to Core Application Service & Workflow Module #5)*
- **FR-06**: Role-based operational analytics dashboard with audit logging and exportable executive reports. *(Mapping: Mapped to Core Application Service & Workflow Module #6)*

5. Non-Functional Requirements

- Availability & Resilience: Target 99.9%+ production uptime with automated health checks and graceful degradation.

- Security & Governance: Enforce Zero-Trust RBAC, encrypted data in transit (TLS 1.3) and at rest (AES-256), and Human-in-the-Loop (HITL) approval gates for high-stakes actions.
- Performance: API P95 response time under 500ms for standard transactional queries and <4s for multi-agent synthesis.
- Healthcare Data Privacy: Strict PHI/PII isolation, immutable audit trails, and zero-downtime backup recovery.

6. Proposed Solution

We propose a modular, cloud-native platform for MedAxis Multi-Specialty Clinics combining a responsive Next.js operator portal, a FastAPI/NestJS service layer, a grounded RAG + domain intelligence engine, and a PostgreSQL + Redis data backbone. Where applicable, reusable architectural patterns from Cynbit Core Services accelerate foundational delivery while preserving strict tenant isolation.

7. Recommended System Architecture

Decoupled 4-tier architecture: (1) Edge WAF & Next.js Presentation Tier, (2) FastAPI/NestJS Transactional & API Tier, (3) LangGraph Agentic & Hybrid Vector RAG Tier with HITL Approval Sandbox, and (4) PostgreSQL + Redis + Immutable Audit Storage Tier.

Architecture Diagram (Mermaid)

```
graph TD
Client["MedAxis Multi-Specialty Clinics Users & Operators"] -->|HTTPS / TLS 1.3| CDN["Cloudflare / AWS WAF & API Gateway"]
CDN --> UI["Next.js + TypeScript Web Portal"]
UI --> API["FastAPI / NestJS Application Core"]
API --> Auth["Zero-Trust RBAC & OAuth2/OIDC"]
API --> Orch["LangGraph Multi-Agent Orchestrator"]
Orch --> ResAgent["Researcher & Extraction Node"]
Orch --> RAG["Vector RAG Index (Pinecone / Semantic Embeddings)"]
Orch --> ArchNode["Domain Logic & Scoring Engine"]
Orch --> CriticNode["Automated Policy & Guardrail Verifier"]
CriticNode -->|High-Stakes Action| HITL["Human-in-the-Loop (HITL) Approval Queue"]
HITL -->|Approved| DB[("PostgreSQL Primary + Read Replica")]
API --> Cache[("Redis Cache & Task Queue")]
API --> Audit[("Immutable Compliance Audit Log")]
```

Key Architectural Decisions

- Event-driven decoupling via Redis Streams prevents slow AI/LLM inference calls from blocking synchronous user HTTP requests.
- Strict separation between retrieved RAG evidence and generated agent proposals prevents ungrounded hallucinations.
- Sandboxed HITL state machine ensures no automated agent can execute destructive or compliance-sensitive operations without human sign-off.

8. Technology Stack

- **Frontend & Client Portal — TypeScript, Next.js (React) & Tailwind CSS**: Listed in Cynbit's Services Hub (https://www.cynbit.in/services); provides SSR/ISR performance, strict type safety, and responsive role-based operational dashboards.
- **Core Backend & API Gateway — Python (FastAPI) & NestJS (TypeScript)**: FastAPI provides async, Pydantic-validated AI/ML model serving and agent orchestration, while NestJS handles modular enterprise RBAC and webhook integrations.

- **AI / Multi-Agent & RAG Layer — LangGraph / LangChain, Gemini / OpenAI API & Pinecone / Hybrid Vector Index**: Enables stateful multi-agent orchestration with bounded retries, semantic retrieval over domain documents, and Human-in-the-Loop (HITL) sandbox governance.
- **Primary Transactional & Caching Data Stores — PostgreSQL (Prisma ORM) & Redis**: PostgreSQL ensures ACID compliance, relational integrity, and row-level multi-tenant security; Redis provides sub-millisecond caching, rate limiting, and distributed task queues.
- **Cloud Infrastructure & DevSecOps — AWS / Azure Zero-Trust, Docker, Kubernetes & Terraform CI/CD**: Delivers reproducible Infrastructure-as-Code, automated container scaling, VPC isolation, and zero-downtime blue/green deployments (99.9% uptime target).

9. Database Design

- `organizations & users` **(PostgreSQL)**: Multi-tenant organization hierarchy, RBAC roles (Admin, Manager, Operator), and SSO metadata. | *Columns:* `id (UUID PK), org_id (UUID FK), email, role, mfa_enabled, created_at`
- `domain_records & workflows` **(PostgreSQL)**: Primary transactional entities for AI Patient Symptom Triage & Clinical Intake Platform with state-machine status tracking and JSONB metadata. | *Columns:* `id (UUID PK), org_id (UUID FK), status, payload_jsonb, version, updated_at`
- `ai_agent_runs & hitl_approvals` **(PostgreSQL)**: Stores agent execution traces, retrieved RAG citation IDs, Critic scores, and human reviewer sign-offs. | *Columns:* `run_id (UUID PK), entity_id (FK), agent_stage, critic_score, hitl_status, reviewer_id, reviewed_at`
- `vector_knowledge_chunks` **(Pinecone / pgvector)**: Stores chunked domain documents, embeddings, and verified source URLs for semantic retrieval. | *Columns:* `chunk_id (PK), doc_id, embedding_vector, source_url, verified_flag`

10. API and Integration Requirements

- `POST /api/v1/workflows/ingest`: Validates and ingests incoming domain records/requests with idempotency key.
- `POST /api/v1/agents/execute`: Triggers stateful Researcher -> Architect/Domain -> Critic pipeline asynchronously.
- `GET /api/v1/workflows/{id}/trace`: Returns step-by-step agent trace, RAG citations, and Critic audit findings.
- `POST /api/v1/hitl/{id}/decision`: Records authenticated human reviewer Approval, Rejection, or Revision Request.
- `GET /api/v1/analytics/summary`: Provides real-time operational KPIs, SLA latency metrics, and audit exports.

11. Relevant Cynbit Case Studies (Verified RAG Evidence)

- **[CS-WIZHEALTH] AI Diagnostic Triage: Enhancing Medical Efficiency** (Client: WizHealth | Similarity: `0.5153` | Verified Metric: *Published on cynbit.in/case-studies (Page header benchmarks: 99.99% Uptime, +140% Growth)* | Source: https://www.cynbit.in/case-studies) — Verified public reference on https://www.cynbit.in/case-studies (Software Development: AI Diagnostic Triage: Enhancing Medical Efficiency).
- **[CS-PRAGYA] IoT Remote Patient Monitoring for Rural Health** (Client: Pragya Healths | Similarity: `0.0947` | Verified Metric: *Published on cynbit.in/case-studies (IoT Remote Patient Monitoring for Rural Health)* | Source: https://www.cynbit.in/case-studies) — Verified public reference on https://www.cynbit.in/case-studies (Software Development: IoT Remote Patient Monitoring for Rural Health).

12. Implementation Phases (Cynbit 6-Stage Lifecycle)

- **Phase 01: Discovery & Audit** (*Weeks 1–2 (Preliminary Assumption)*): Stakeholder requirements workshops, legacy data/API audit, and HITL governance boundary sign-off.
- **Phase 02: Strategy & Blueprinting** (*Weeks 2–3 (Preliminary Assumption)*): Finalized OpenAPI specs, PostgreSQL ER diagrams, cloud security topology, and UI wireframes.
- **Phase 03: High-Fidelity Design** (*Weeks 3–4 (Preliminary Assumption)*): Interactive Next.js design system, operator workflows, and HITL review console prototypes.
- **Phase 04: Technical Implementation** (*Weeks 5–16 (Preliminary Assumption)*): Core FastAPI/NestJS services, PostgreSQL migrations, RAG/AI pipeline integration, and frontend build.
- **Phase 05: Quality & Integrity Check** (*Week 17 (Preliminary Assumption)*): Automated Pytest suite, load/stress testing (99.9% uptime target), security red-teaming, and UAT.
- **Phase 06: Deployment & Evolution** (*Week 18 (Preliminary Assumption)*): Zero-downtime production launch on AWS/Azure, operator training, and telemetry SLA monitoring.

13. Preliminary Timeline and Effort Estimates

Disclaimer: *PRELIMINARY ASSUMPTION NOTICE: All timeline and effort figures below are non-binding preliminary engineering assumptions generated for discovery scoping. Final delivery commitments, pricing, and resource allocation require formal Senior Architect scoping with the customer.*

- **Estimated Duration Range:** 18–22 Weeks (Preliminary Engineering Assumption)
- **Assumed Team Composition:** 1 Senior Technical Architect (Part-Time), 2 Full-Stack/Backend Engineers, 1 AI/ML Engineer, 1 QA/DevSecOps Engineer (Preliminary Assumption).
- **Primary Effort Drivers:** 6 functional capability modules, 4 non-functional SLA targets, and external integration validation.

14. Infrastructure Requirements

- Compute: Containerized Docker services orchestrated via managed Kubernetes (EKS/AKS) or AWS ECS Fargate with auto-scaling.
- Database & Cache: Multi-AZ managed PostgreSQL 16 with automated point-in-time recovery (PITR) and managed Redis 7 cluster.
- CI/CD & Observability: GitHub Actions + Terraform Infrastructure-as-Code with Prometheus/Grafana and structured JSON logging.

15. Security Considerations

- Zero-Trust Role-Based Access Control (RBAC) with OAuth2/OIDC and short-lived JWT session tokens.
- Encryption in transit (TLS 1.3) and at rest (AES-256) across PostgreSQL volumes and S3 object buckets.
- Human-in-the-Loop (HITL) Sandbox Gate: Autonomous AI agents draft recommendations and stage actions, while high-stakes mutations require explicit human operator sign-off.
- Input Sanitization & Prompt-Injection Firewall: Strict Pydantic schema validation, token ceiling enforcement, and system-prompt isolation.
- Immutable Audit Logging: Every agent invocation, retrieval source, and human approval action is logged with actor ID and UTC timestamp.

16. Technical Risks and Mitigation

- **Third-Party API Schema Drift or Rate Limiting (Severity: Medium)**: Implement circuit breakers, exponential backoff, Redis request queuing, and strict Pydantic schema validation.

- **AI / LLM Hallucination or Ungrounded Recommendations (Severity: High)**: Enforce strict RAG citation thresholds, deterministic Critic Agent verification, and mandatory Human-in-the-Loop sign-off.
- **Legacy Data Quality & Migration Inconsistencies (Severity: Medium)**: Execute staged dry-run ETL migrations in a sandbox environment with automated reconciliation checksums before cutover.

17. Assumptions and Missing Information

Key Engineering Assumptions

- Customer will provide timely access to domain subject-matter experts (SMEs) and sandbox API credentials during Phase 01.
- All production deployments will utilize standard AWS or Azure cloud regions compliant with customer data residency rules.

Missing Information to Clarify in Discovery Call

- Specific third-party APIs, identity providers (SSO/OAuth), and legacy database schemas need confirmation during Discovery.
- Peak concurrent user volume and regional data-residency compliance jurisdictions require formal verification.

18. Recommended Next Steps

- Step 1: Internal Cynbit Senior Architect reviews and approves this AutoBlueprint in the Approval Center.
- Step 2: Conduct 45-minute technical discovery call with the customer to resolve the Missing Information items in Section 17.
- Step 3: Finalize Phase 01 Discovery & Audit statement of work and provision isolated sandbox environment.